

Agent Script

The .agent DSL for authoring Agentforce agents in Agentforce DX.

What Is Agent Script

Agent Script is the language for building **Agentforce** agents in Agentforce Builder & DX. It blends **natural-language** instructions with **programmatic logic** – `if/else`, transitions, variables – for deterministic business rules.

- GA since Feb 2026 (beta Dec 2025).
- Source lives in a `.agent` file, packaged as `AiAuthoringBundle` metadata.

File & Bundle

```
aiAuthoringBundles/  
My_Agent/  
  My_Agent.agent           // script  
  My_Agent.bundle-meta.xml // config
```

- `AiAuthoringBundle` metadata – **API v65.0+**.
- `bundleType: AGENT`; set `target: {Bot}`. `{BotVersion}` to **commit** a version (omit it = draft).

Top-Level Blocks

BLOCK	PURPOSE
<code>system:</code>	global instructions + <code>messages:</code>
<code>config:</code>	developer_name, agent_type, role
<code>language:</code>	default + additional locales
<code>variables:</code>	globals shared by subagents
<code>connection:</code>	channels (e.g. messaging)
<code>start_agent</code>	the router / entry point
<code>subagent</code>	one per subagent (repeatable)

Anatomy

```
variables:  
  isPremium: mutable boolean = false  
  
start_agent hello:  
  description: "Greet the user."  
  reasoning:  
    instructions: ->  
      if @variables.isPremium:  
        | offer to redeem Premium points  
      else:  
        | offer a Premium upgrade
```

Subagents

- A **subagent** (formerly **Topic**) is a job area with a `description`, `reasoning`, and local `actions`.
- The router reads `description` to decide when it applies.
- `before_reasoning:` / `after_reasoning:` run logic every turn.
- The `topic` keyword is **deprecated** – use `subagent`.

Start Agent (Router)

```
start_agent router:  
  reasoning:  
    actions:  
      to_orders: @utils.transition to  
                  @subagent.Orders  
                  available when @variables.verified
```

- Entry point for **every turn**; classifies and routes.
- Gate each option with `available when`.

Reasoning

- `instructions: ->` mixes **logic** (`if`, `run`, `set`) with **prompt** lines prefixed `|`.
- Interpolate values into prompts with `{!@variables.x}`.
- `actions:` lists the **tools** the LLM may choose to call.

Actions

```
actions:  
  lookup_order:  
    description: "Retrieve an order."  
    inputs:  
      query: string  
    outputs:  
      summary: string  
    target: "flow://GetOrdersByContact"
```

- Target schemes: `flow://`, `apex://`, `prompt://`.
- Run it: `run @actions.lookup_order with query = @variables.email`, then `set @variables.summary = @outputs.summary`.

Variables

- Regular** – `name:` [mutable] `type = default`.
- Linked** – `name:` linked `type` with a `source:` (e.g. `@session.sessionID`).
- System** – read-only, e.g. `@system_variables.user_input`.
- Reference as `@variables.x` (logic) or `{!@variables.x}` (prompt text).
- Types: `string`, `number`, `boolean`, `object`, `date`, `id`, `list[type]`.

Flow of Control

- `if` / `else` only – there is no `else if`.
- Operators: `==`, `!=`, `<`, `<=`, `>`, `>=`, `is`, `is not`, `and`, `or`, `not`, `+`, `-`.
- `transition to @subagent.X` – one-way; the target restarts from the top.
- Reference `@subagent.X` as a **tool** to *consult* it and return.
- `@utils.escalate` hands off to a human via a messaging connection.

Grounding

- Agent Script has **no** `grounding:` block.
- Assign a **Data Library** (RAG over Knowledge / files) to the agent in **Agentforce Builder**.
- The script then consumes grounded results through ordinary **actions** and **variables**.

CLI Workflow

```
sf agent generate agent-spec  
sf agent generate authoring-bundle \  
  --spec spec.yaml --api-name My_Agent  
sf agent validate authoring-bundle \  
  --api-name My_Agent  
sf agent preview --api-name My_Agent \  
  --use-live-actions  
sf agent publish authoring-bundle \  
  --api-name My_Agent
```

- Needs **sf CLI ≥ 2.123.1**. `sf agent create` is the **legacy**, non-Agent-Script path.

Testing

```
sf agent generate test-spec --output-file  
spec.yaml  
sf agent test create --spec spec.yaml  
sf agent test run --api-name My_Suite \  
  --result-format junit --wait 10
```

- Suites are `AiEvaluationDefinition` metadata (**API v63.0+**).
- Asserts `topic_sequence_match`, `action_sequence_match`, `bot_response_rating`, `coherence`.

Deploy & Activate

- `sf agent publish authoring-bundle` compiles the `.agent` and creates **Bot / BotVersion / GenAi*** metadata.
- Ship the bundle: `sf project deploy start --metadata AiAuthoringBundle:My_Agent`.
- Go live and monitor with `sf agent activate` / **Agentforce Studio**.
- Session Tracing** streams production runs into **Data 360**.

Metadata Map

TYPE	REPRESENTS
<code>Bot</code> / <code>BotVersion</code>	the agent + version
<code>GenAiPlannerBundle</code>	the reasoning planner
<code>GenAiPlugin</code>	a subagent
<code>GenAiFunction</code>	an action
<code>GenAiPromptTemplate</code>	a prompt template